Benchmarking Keyword Spotting Architectures across TensorFlow Lite and ONNX Runtime on a Raspberry Pi 5

Fatih Jawwad Al Mumtaz · Amirul Al Aziz

Faculty of Science and Technology, Universitas Darussalam Gontor, Indonesia

SUPPORTED BY

Outline







1 Background & motivation — why runtime choice matters for always-on voice

2 Models & method — four architectures, one frontend, rigorous protocol

3 Results — accuracy, latency, threads, energy, and the trade-off

4 Conclusion — what to pick, and why habit is not evidence

SUPPORTED BY

Why benchmark KWS on the Pi 5?







- Keyword spotting listens all day for a word or two — on a small board that means sub-millisecond latency and milliwatt power, with no internet to fall back on.

Everyone assumes TensorFlow Lite is fastest on Arm. That assumption is rarely tested head-to-head on 64-bit Arm, across CNNs, dense nets, and RNNs.

- On-device voice also matters for care settings: no cloud means usable in low-connectivity clinics and no health-speech privacy risk (SDG 3).

Question: does the default runtime deserve its default status?

SUPPORTED BY
IEEE INDONESIA SECTION
AiMed Artificial Intelligence-Medical
comnets
RIS RESEARCH AND INNOVATION SERVICES

What we test







Four models, three configs, one board: DS-CNN (22.6k) · DNN (88.4k) · LSTM (20.0k) · CRNN (56.8k) × TFLite fp32 / TFLite int8 / ONNX fp32, all on a Raspberry Pi 5.

Same everything else: identical 49×10 MFCC frontend, same 12-way head, same clips, same protocol.

Three contributions: (1) full training + export pipeline incl. fp32 reconstruction; (2) accuracy + latency on both tasks, plus thread sweep and PMIC energy; (3) proof the runtime gap does not generalize.

SUPPORTED BY

Four models, one frontend

Model	Params	Structure
DS-CNN (MLPerf ref.)	22,604	10×4 conv (stride 2) + 4× depthwise/pointwise blocks + GAP + dense
DNN baseline	88,396	490–128–128–64–12 dense, dropout 0.1 (lower-bound anchor)
LSTM	19,980	LSTM-64 unrolled over 49 steps + dense
CRNN	56,844	2× 3×3 conv + 2× maxpool + GRU-64 + dense

- Same 49×10 MFCC in, same 12-way softmax out. DNN is deliberately the anchor — dense-only is not competitive at this budget, its low score is expected, not a bug.

SUPPORTED BY

The reference artifact was not fp32

Found: the official MLPerf kws_ref_model artifact is mixed-precision — int8 conv kernels, fp32 rest. Benchmarking it as "fp32" would poison the comparison.

Rebuilt: enumerated every tensor via the LiteRT API, dequantized kernels with $\hat{w} = (q - z)\cdot s$, reassembled 9 fused conv–bias–ReLU blocks + GAP + dense in Keras.

Verified: max abs diff 7×10^{-3} on softmax, 100% argmax agreement over 1,000 frames — then exported clean fp32 TFLite + fp32 ONNX of identical structure.

All three artifacts (original, rebuilt TFLite, rebuilt ONNX) are benchmarked.

Full-int8 quantization, done properly

- New full-int8 DS-CNN via post-training quantization, calibrated on 1,000 training clips (seed 42) — 25% faster than the mixed-precision artifact at equal accuracy.
- Keras 3 emits dynamic tensor-list loops for RNNs that the TFLite converter cannot lower — exported with unroll=True, an identical-weight static graph both runtimes accept.

Cost is uneven: CNNs lose < 0.5 pt to int8; the LSTM loses 3.05 pts (subset) / 4.12 pts (full task) — recurrence wants quantization-aware training.

How we measured

Board: Pi 5 (BCM2712, 4× A76 @ 2.4 GHz), 8 GB RAM · Debian 64-bit · governor pinned to performance · LiteRT 2.1.6 vs ONNX Runtime 1.28.

Stats: 1,000 warmup + 1,000 timed passes, sequential, features precomputed · IQR outlier filter · 10k bootstrap 95% CIs.

Data: 4,074 clips (10 commands) + 12,005 clips (full 12-class) · thread sweep at 1/2/4 threads · PMIC energy (SoC only; DNN omitted, < 0.1 ms too fast to sample).

Accuracy holds up — except LSTM

Model	fp32	int8	Δ	fp32	int8	Δ
DS-CNN	90.55%	90.57%	+0.02	95.49%	95.21%	−0.28
DNN	79.63%	79.65%	+0.02	72.65%	71.90%	−0.75
LSTM	90.48%	87.43%	−3.05	88.84%	84.72%	−4.12
CRNN	92.56%	92.17%	−0.39	92.08%	91.60%	−0.48

- Left block: 10 commands (4,074 clips) · right block: full 12-class (12,005 clips). TFLite fp32 and ONNX fp32 agree on all clips (diff < 3×10^{-6}) — runtimes are numerically interchangeable, speed is the whole story.

Latency flips by architecture

Model	TFLite fp32	TFLite int8	ONNX fp32	ONNX / TFLite
DS-CNN	0.211 ms	0.127 ms	0.170 ms	0.81 — ONNX wins 19.2%
DNN	0.016 ms	0.007 ms	0.029 ms	1.81× — TFLite wins
LSTM	0.372 ms	0.157 ms	0.778 ms	2.09× — TFLite wins
CRNN	0.259 ms	0.102 ms	0.361 ms	1.40× — TFLite wins

int8 is 1.7–2.5× faster almost for free · ONNX wins only on depthwise convs.

SUPPORTED BY

Threads don’t flip it — and faster means cooler

Config	T=1	T=2	T=4
DS-CNN TFLite	0.212	0.123	0.078
DS-CNN ONNX	0.438	0.259	0.169
CRNN TFLite	0.260	0.187	0.220
CRNN ONNX	0.508	0.419	0.361

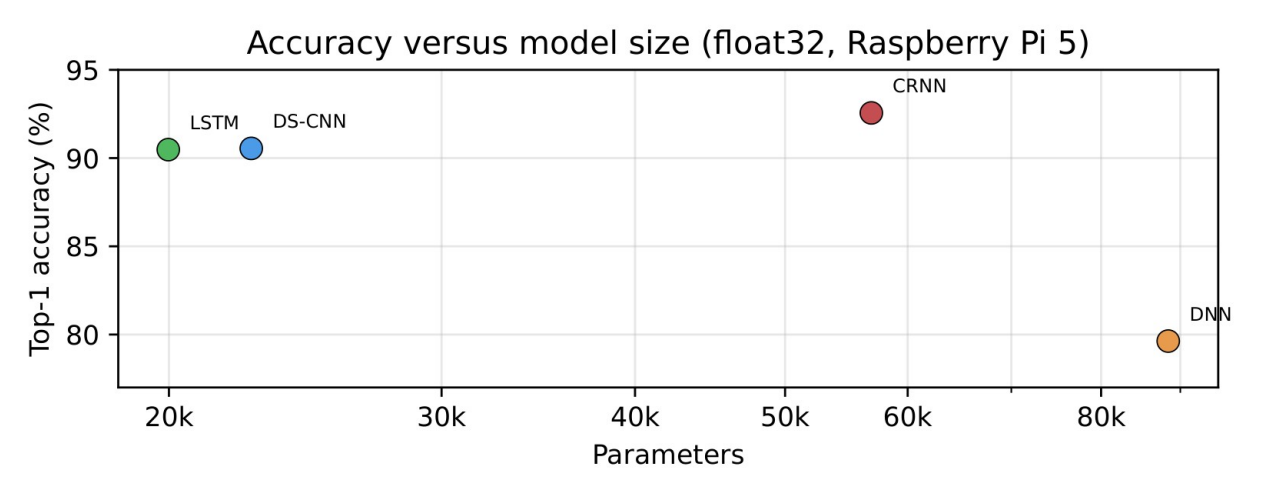
Model	PΔ (W)	t (ms)	E (mJ)
DS-CNN TFLite	1.59	0.22	0.34
DS-CNN ONNX	1.18	0.45	0.53
CRNN TFLite	1.85	0.27	0.51
CRNN ONNX	1.58	0.49	0.78

- Default ONNX quietly spawns 4 threads while TFLite runs 1 — pinned equally, TFLite still wins everywhere.
- Race-to-sleep wins: TFLite draws more power yet finishes ~2× faster, so energy drops ~35% (0.34 vs 0.53 mJ).

SUPPORTED BY

The 2nd International Conference on Artificial Intelligence and Technological Solutions
"Artificial Intelligence-Driven Innovations and Partnerships for Health, Clean Water, and Sustainable Development"
in conjunction with The 14th ASEAN Workshop on Information Science and Technology (AWIST) and The 8th Annual Research Seminar (ARS)
icaitech.org | icaitech@unsri.ac.id

So which model do you ship?



Sweet spot: DS-CNN — 22.6k params, 95.5% full-task, 0.13 ms in int8.

Most accurate: CRNN — 92.6% subset at 56.8k params, 0.10 ms int8.

- LSTM is small but int8-unfriendly; DNN is just the anchor — big and weak.

Rule: pick the runtime by operator mix, not by habit.

Takeaways







Test both runtimes before you commit: ONNX wins only on DS-CNN depthwise convs; TFLite wins 1.4–2.1× on dense and recurrent.

Pin your threads: defaults lie — ONNX spawns 4, TFLite runs 1.

- Full int8 is nearly free for CNNs (1.7–2.5× faster, < 0.5 pt drop); LSTM needs QAT.
- Latency saves battery: finishing 2× faster cuts energy by about a third.

SUPPORTED BY

Thanks — questions welcome

Artifact, code, and data are open

github.com/jaweed3/kws-pi5-benchmark

DOI 10.5281/zenodo.23765521 · AGPLv3

Fatih Jawwad Al Mumtaz · fatihalmumtaz76@student.cs.unida.gontor.ac.id
Amirul Al Aziz · Faculty of Science and Technology, Universitas Darussalam Gontor

SUPPORTED BY